1. Dai File System ai Database Management Systems (DBMS)

I **File System** tradizionali presentano limitazioni strutturali fondamentali: **isolamento**, **duplicazione** e **rigidità** nella gestione e reperimento dei dati. Per superare questi limiti è stato introdotto il **DBMS**.

Cos'è un DBMS?

Un DBMS (*Database Management System*) **non è il database**, ma il **software** specializzato che consente la creazione, manipolazione, gestione e controllo dei dati all'interno di uno o più database.

Modello Logico vs. Modello Fisico

Un DBMS opera su due distinti livelli di astrazione:

- **Livello Logico:** Rappresenta la struttura dei dati così come viene vista ed elaborata dall'utente/programmatore (es. tabelle, campi, chiavi).
- **Livello Fisico:** Corrisponde alla reale memorizzazione binaria e all'organizzazione dei dati sui dispositivi di archiviazione secondaria (disco rigido, SSD).

Vantaggi Fondamentali dell'uso di un DBMS

1. **Riduzione della Ridondanza e Normalizzazione:** Minimizza la duplicazione inutile dei dati tramite tecniche di normalizzazione (tecnica analizzata dopo la modellazione logica per identificare e rimuovere anomalie e inconsistenze).
2. **Integrità dei Dati:** Garantisce la qualità e l'accuratezza dei dati tramite l'applicazione di vincoli di integrità:
    a. **Vincoli di Dominio:** Limitano i valori che un singolo campo può assumere (es. tipo di dato, intervallo numerico).
    b. **Vincoli di Integrità Referenziale:** Regolano le relazioni tra tabelle diverse tramite l'uso di chiavi esterne (*Foreign Key*). Impediscono l'esistenza di dati "orfani" (es. un ordine associato a un ID cliente inesistente).
    c. **Azioni Referenziali (es. `ON DELETE CASCADE`):** Regole che definiscono il comportamento del sistema quando un dato padre viene eliminato o aggiornato.
3. **Controllo della Concorrenza e Transazioni:** Gestisce l'accesso simultaneo di più utenti agli stessi dati senza causare conflitti o corruzioni.

    a. **Transazione:** Blocco di istruzioni SQL eseguito in maniera **atomica** (tutto o niente).
    b. Le transazioni devono rispettare le proprietà **ACID**: *Atomicità*, *Consistenza*, *Isolamento* (*Isolation*), *Durabilità*.
4. **Sicurezza e Controllo degli Accessi:** Gestione granulare dei permessi tramite **ruoli** e **privilegi** (es. consentire a un gruppo di utenti la sola operazione di lettura e ad altri anche la scrittura).
5. **Efficienza Temporale e Spaziale:**
    a. **Temporale:** Utilizzo di indici e ottimizzatori di query per restituire risultati in pochi millisecondi rispetto alle ricerche manuali su file o archivi cartacei.
    b. **Spaziale:** Ottimizzazione dell'occupazione del disco fisso.
6. **Backup e Recovery:** Strumenti integrati per il ripristino dei dati in caso di guasti hardware o disastri.

Esempio Pratico: Gestione della Concorrenza vs. Overbooking

- **Gestione Corretta con Transazioni (es. Biglietteria Eventi):**

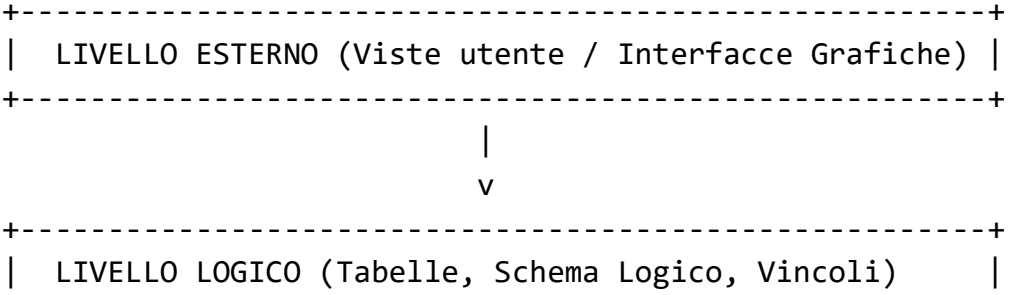
Quando un utente seleziona un posto su TicketOne, il DBMS avvia una transazione e imposta un *countdown*. Il posto viene temporaneamente "bloccato" (isolato da altri utenti). Se l'acquisto va a buon fine, la transazione fa il *commit*; in caso contrario, fa il *rollback* e libera la risorsa.

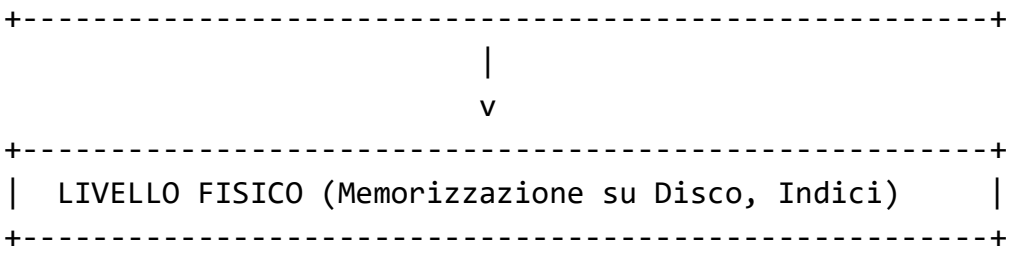
- **Gestione Senza Transazioni Strette (es. Overbooking Aereo):**

Le compagnie aeree sfruttano di proposito la concorrenza aperta senza blocchi rigidi per vendere più biglietti dei posti reali sul volo, basandosi sulla probabilità statistica che alcuni passeggeri non si presentino al gate (*no-show*).

2. Architettura ANSI/SPARC

Standard teorico proposto nel 1975 che definisce un'architettura a **tre livelli di astrazione** per disaccoppiare le applicazioni utente dalla struttura fisica dei dati.





Concetto di Indipendenza dai Dati

1. **Indipendenza Logica:** La capacità di modificare lo schema logico (es. aggiungere una colonna a una tabella) senza dover modificare le viste esterne o i programmi applicativi che non usano direttamente quel campo.
2. **Indipendenza Fisica:** La capacità di modificare la struttura fisica di memorizzazione (es. riorganizzare i file su disco, modificare gli indici di ricerca) senza dover alterare lo schema logico.

3. Categorie di Utenti di un DBMS

I soggetti che interagiscono con un sistema di basi dati si dividono in:

- **Database Administrator (DBA):** Responsabile della configurazione, ottimizzazione delle prestazioni, sicurezza, definizione di routine/job pianificati e strategie di backup.
- **Progettisti e Sviluppatori:** Progettano gli schemi (concettuale e logico) e sviluppano il codice delle applicazioni che interagiscono con il DB.
- **Analisti:** Definizione dei requisiti aziendali e modellazione preliminare.
- **Utenti Finali:** Utilizzatori dell'applicazione applicativa (es. un cliente web) che interagiscono in modo trasparente con il DB tramite interfacce grafiche.

4. Tipologie di DBMS Non Relazionali (NoSQL)

A differenza dei DBMS relazionali (RDBMS) basati su tabelle, i sistemi NoSQL utilizzano strutture dati flessibili:

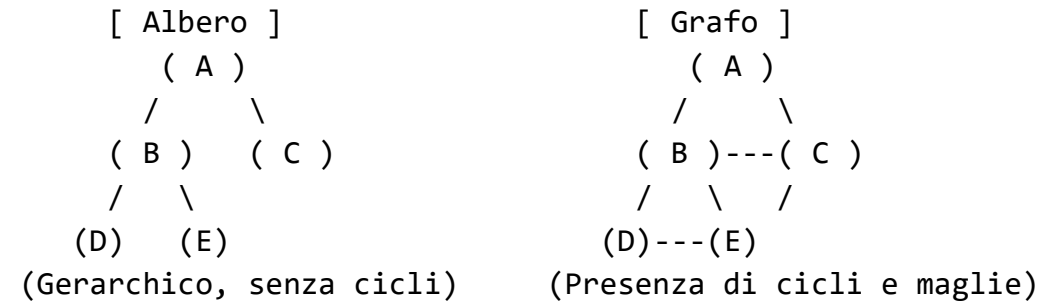
- **Documentali (es. MongoDB):** I dati sono strutturati come documenti flessibili (tipicamente in formato **JSON** o BSON). Non richiedono uno schema rigido preventivo: documenti appartenenti allo stesso insieme possono avere campi diversi.
- **Column-Oriented / Column-Family (es. Apache Cassandra):** Progettati per la gestione scalabile di enormi moli di dati (*Big Data*) memorizzando le informazioni per colonne anziché per righe.

- **A Grafo (es. Neo4j):** Basati sulla teoria dei grafi per modellare dati ad alta interconnessione.

Esempio Pratico: Strutture Dati a Grafo vs. Albero

In informatica, alberi e grafi sono strutture dati non lineari:

- **Albero:** Struttura gerarchica in cui ogni nodo ha un solo nodo padre (tranne la radice). Non sono ammessi cicli.
- **Grafo:** Insieme di nodi (vertici) e archi (collegamenti). Può contenere **cicli** e un nodo può avere più connessioni/padri.



I DB a Grafo (come Neo4j) sono ideali per social network (es. "amico di un amico") o sistemi di raccomandazione.

5. Modellazione Concettuale: Il Modello Entity-Relationship (ER)

Definizione Matematica di Relazione

Dal punto di vista matematico, una **Relazione** R tra due insiemi di entità A e B è un **sottoinsieme del prodotto cartesiano** A \ B.

Esempio Pratico: Prodotto Cartesiano e Relazione

Dati due insiemi:

- A = {A_1, A_2} (Insieme degli Studenti)
- B = {B_1, B_2, B_3} (Insieme dei Corsi)

l prodotto cartesiano A × B rappresenta tutte le combinazioni possibili:

$A \times B = \{(A_1, B_1), (A_1, B_2), (A_1, B_3), (A_2, B_1), (A_2, B_2), (A_2, B_3)\}$

La relazione reale nel database è un sottoinsieme di questo prodotto cartesiano, poiché un singolo studente non è iscritto contemporaneamente a tutti i corsi esistenti, ma solo ad alcuni di essi:

$R = \{(A_1, B_1), (A_2, B_3)\} \subseteq A \times B$

Nelle occorrenze di una relazione non possono esistere tuple (coppie o n-ple) duplicate.

Componenti del Diagramma ER

1. Entità e Relazioni

- **Entità:** Rappresentano classi di oggetti omogenei del mondo reale (es. `Studente`, `Corso`).
- **Relazione/Associazione:** Legame logico tra due o più entità.

Ristrutturazione Modello: Se una relazione semplice non è sufficiente a modellare un problema (es. uno studente che tenta lo stesso esame in più appelli differenti), la relazione viene reificata, ossia trasformata in un'**entità intermedia** (es. l'entità `Sostenimento` o `Appello`).

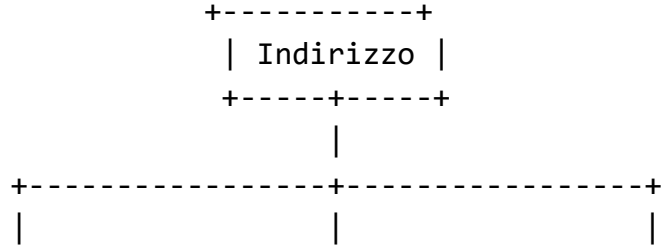
2. Attributi

Caratteristiche elementari delle entità o delle relazioni.

- **Semplici:** Valore atomico (es. `Nome`, `Cognome`, `Età`). Si disegnano con un pallino vuoto.
- **Composti:** Attributi suddivisi in sotto-proprietà più specifiche per facilitarne la gestione logica.

Esempio di Attributo Composto

Invece di definire un attributo unico `Indirizzo` (stringa generica), si definisce un attributo composto costituito da un insieme di attributi semplici:



```
   v                  v                  v
( Via )          ( Civico )         ( CAP )
```

- **Multi-valore:** Attributi che possono contenere più di un valore per una singola occorrenza (es. `Telefono` con cardinalità `(0,N)`).

3. Identificatori (Chiavi Candidate)

Rappresentano l'insieme di attributi che identificano in modo univoco un'occorrenza di un'entità. Si rappresentano graficamente con un **pallino pieno**.

- **Identificatori Interni:**
  - **Semplici:** Un singolo attributo (es. `Matricola` o `Email`).
  - **Composti:** Un'unione di due o più attributi collegati tra loro (es. Nome + Cognome).
- **Identificatori Esterni:** Utilizzano attributi di altre entità collegate tramite relazioni (analizzati nelle fasi successive di progettazione).

Nota: Le **relazioni non hanno identificatori propri**; la loro univocità è data dagli identificatori delle entità che vi partecipano.

Ecco la versione riscritta senza utilizzare i simboli di dollaro o la notazione matematica/LaTeX, così da poterla copiare e incollare senza problemi di formattazione:

1. Concetti Fondamentali del Modello E-R (Entità-Relazioni)

Entità e Occorrenze

- **Entità**: Rappresenta una classe o categoria di oggetti omogenei del mondo reale (ad esempio STUDENTE, CORSO). Va indicata sempre con il nome al **singolare**.
- **Occorrenza (o Istanza)**: È il singolo elemento appartenente al gruppo identificato dall'entità (ad esempio lo studente "Mario Rossi" è una singola occorrenza dell'entità STUDENTE).
  - *Rappresentazione insiemistica*: Se consideriamo l'entità come un insieme matematico, le occorrenze sono i singoli punti o elementi contenuti al suo interno.

Attributi e Identificatori (Chiavi)

- **Attributo**: Una caratteristica o proprietà che descrive l'entità (ad esempio Nome, Cognome, Matricola).
- **Identificatore Univoco (Chiave Candidata)**: Un attributo (o insieme di attributi) che permette di distinguere e **identificare univocamente** ogni singola occorrenza di un'entità.
    - **Identificatore Semplice**: Costituito da un solo attributo (ad esempio Matricola).
    - **Identificatore Composto**: Costituito da più attributi aggregati insieme (ad esempio l'unione di Nome e Cognome, se ritenuti univoci nel contesto).
    - **Fase Logica**: Nel modello concettuale E-R si individuano i possibili identificatori candidati. Successivamente, nella progettazione logica del database, tra questi verrà scelta la **Chiave Primaria (Primary Key)** effettiva.

2. Identificatori nelle Relazioni (Associazioni)

Perché le relazioni NON hanno un proprio Identificatore (ID)?

Un'associazione (relazione) tra due entità descrive il legame tra le rispettive occorrenze:

- Matematicamente, un'associazione è un **sottoinsieme del prodotto cartesiano** tra gli insiemi delle occorrenze delle entità coinvolte.
- **Esempio**: L'associazione ESAME collega le occorrenze dell'entità STUDENTE con quelle dell'entità CORSO. Un'occorrenza della relazione sarà una coppia formato dallo studente 1 e dal corso 1.
- **Univocità della relazione**: La combinazione degli identificatori delle entità partecipanti (ad esempio Matricola Studente + Codice Corso) identifica già in modo univoco ogni singola riga o coppia della relazione. Aggiungere un ulteriore ID proprio alla relazione sarebbe ridondante.
- **Nota**: Se si presentasse la necessità strutturale di assegnare un identificatore proprio a un'associazione, questa va concettualmente **trasformata in un'Entità**.

Relazioni Ricorsive

Il medesimo principio del prodotto cartesiano si applica anche alle **relazioni ricorsive** (relazioni di un'entità con se stessa): un'occorrenza sarà composta da una coppia di

istanze distinte appartenenti alla stessa entità (ad esempio lo studente 1 legato allo studente 2).

3. Cardinalità delle Relazioni

- **Cardinalità di un'Entità**: In ambito insiemistico indica il numero totale di occorrenze appartenenti a quell'entità.
- **Cardinalità di una Relazione (Molteplicità)**: Esprime il numero minimo e massimo di occorrenze di una relazione a cui un'occorrenza di un'entità può partecipare. Si rappresenta con coppie del tipo (min, max):
    - **Minimo (0 o 1)**: Indica la partecipazione facoltativa (0) o obbligatoria (1).
    - **Massimo (1 o N)**: Indica se il legame è singolo (1) o multiplo (N).

Lettura delle Cardinalità

Nelle relazioni binarie la cardinalità si valuta ed esamina nei due versi:

1. **Da Entità A verso Entità B**: Quante occorrenze di B possono corrispondere a una singola occorrenza di A
2. **Da Entità B verso Entità A**: Quante occorrenze di A possono corrispondere a una singola occorrenza di B

(Esempio: Uno STUDENTE può sostenere da 0 a N ESAMI; un CORSO può essere sostenuto da 0 a N STUDENTI).

1. Cardinalità nelle Relazioni Ternarie (e N-arie)

Concetto e Differenza dalle Relazioni Binarie

- Nelle **relazioni binarie**, la cardinalità si legge considerando una singola entità verso l'altra (es. da Studente a Corso e da Corso a Studente).
- Nelle **relazioni ternarie** (o n-arie), che coinvolgono tre o più entità contemporaneamente, la cardinalità viene determinata **fissando una coppia di entità** e valutando con quante occorrenze della terza entità quella coppia può associarsi.

Esempio: Relazione Ternaria tra Studente, Corso e Aula

Ipotizziamo una relazione che unisce le tre entità `STUDENTE`, `CORSO` e `AULA`.

- **Lettura della cardinalità**: Presa una coppia specifica (`STUDENTE`, `CORSO`), ci si chiede: *in quante AULE si può svolgere quell'esame per quel determinato studente?*
- **Determinazione dei Limiti (Minimo e Massimo)**:
    - **Valore Massimo (1 oppure N)**: Se le specifiche del problema stabiliscono che un esame di un determinato corso per uno studente può svolgersi in una sola aula, il massimo sarà **1**. Se invece potesse svolgersi in più aule, il massimo sarebbe **N**.
    - **Valore Minimo (0 oppure 1)**: Se l'esistenza della coppia `STUDENTE-CORSO` richiede obbligatoriamente un'aula assegnata, il minimo sarà **1**. Se l'esame può essere registrato senza ancora aver assegnato un'aula (es. esame a distanza o aula non ancora definita), il minimo sarà **0**.

2. Importanza delle Specifiche del Dominio (Requisiti)

Nessuna cardinalità è "giusta" o "sbagliata" a priori: **dipende sempre dalle regole di business (requisiti)** fornite dal cliente o dal testo del problema.

- **Esempio dell'Aula**:
    - Se il regolamento dell'università stabilisce che in un'aula può svolgersi un solo esame per volta o che la combinazione esame-studente richiede un'unica aula, la cardinalità sarà **(1,1)**.
    - Se il regolamento permette soluzioni differenti o flessibili, la cardinalità può cambiare in **(0,1)** o **(1,N)**.

3. Secondo Esempio: Relazione Ternaria "Acquisto"

Consideriamo una relazione ternaria `ACQUISTO` (o transazione) che collega le entità `CLIENTE`, `PRODOTTO` e `FATTURA`.

Analisi delle Cardinalità

1. **Fissando la coppia (CLIENTE, PRODOTTO)**:
    a. Quante fatture possono essere associate a un determinato cliente che acquista un determinato prodotto?
    b. Risposta tipica: **1** (l'acquisto genererà un'unica fattura specifica).
2. **Fissando la coppia (CLIENTE, FATTURA)**:
    a. Quanti prodotti possono essere associati a una specifica fattura di un determinato cliente?

    b. Risposta tipica: Da **1 a N** (in una singola fattura/transazione il cliente può acquistare più prodotti).
3. **Valutazione del Minimo (0 oppure 1)**:
    a. Esistono prodotti nel catalogo che non sono mai stati acquistati da alcun cliente? Sì, quindi la cardinalità minima verso l'acquisto per un prodotto a catalogo può partire da **0**.

Suggerimento Metodologico per lo Studio

Se il ragionamento diretto sulle relazioni ternarie risulta complesso:

1. Scomponi temporaneamente il problema analizzando le **relazioni binarie** tra le coppie di entità.
2. Fai sempre riferimento al **testo dei requisiti**: quando un dubbio rimane aperto tra due cardinalità diverse (es. tra 0 e 1 per il minimo), la scelta finale deve essere motivata basandosi sulle regole del sistema che si sta progettando.

Ecco la trascrizione e rielaborazione del testo in un formato chiaro, strutturato e privo di simboli matematici speciali o caratteri LaTeX, pronto per essere copiato e incollato.

1. Cardinalità Minima: Partecipazione Obbligatoria vs Opzionale

La **cardinalità minima** (il primo valore della coppia, posizionato a sinistra) definisce il tipo di partecipazione di un'entità a una relazione:

- **Partecipazione Opzionale (Minimo = 0)**:
    - Indica che un'occorrenza dell'entità può esistere anche senza partecipare alla relazione.
    - *Esempio (Aule)*: Se l'entità AULA ha una cardinalità minima pari a **0** nella relazione ESAME, significa che nell'insieme delle aule può esistere un'aula in cui al momento non è programmato alcun esame.
- **Partecipazione Obbligatoria (Minimo = 1)**:
    - Indica che ogni occorrenza dell'entità deve partecipare **almeno una volta** alla relazione.
    - *Esempio (Aule)*: Se l'entità AULA ha una cardinalità minima pari a **1**, ogni singola aula presente nel sistema deve essere associata ad almeno un esame.

2. Cardinalità Massima: Limite delle Occorrenze (1 vs N)

La **cardinalità massima** (il secondo valore della coppia, posizionato a destra) stabilisce il numero massimo di volte in cui una combinazione di elementi può comparire all'interno della relazione:

- **Cardinalità Massima = 1**:
  - Limita la presenza a un'unica associazione.
  - *Esempio (Cliente, Prodotto, Fattura)*: Se la cardinalità massima per FATTURA è **1**, la coppia (Cliente "Luca", Prodotto "Cuffie") può comparire associata alla fattura F1 **una sola volta** all'interno dell'insieme della relazione. Non possono esistere altre righe o tuple con la stessa combinazione legate a fatture diverse.
- **Cardinalità Massima = N**:
  - Permette a un'occorrenza di ripetersi più volte all'interno della relazione.
  - *Esempio*: Permette di associare più prodotti o più elementi alla medesima entità all'interno della relazione.

3. Schema Riassuntivo delle Cardinalità

Elemento	Posizione	Valori Possibili	Significato Concettuale
Partecipazione Minima	Sinistra: **(MIN, max)**	**0** oppure **1**	**0**: Opzionale (può non partecipare) **1**: Obbligatoria (deve partecipare almeno una volta)
Partecipazione Massima	Destra: **(min, MAX)**	**1** oppure **N**	**1**: Univoca (al massimo un'associazione) **N**: Multipla (può partecipare N volte)

Una **relazione ricorsiva** (o *anello*) nei diagrammi Entità-Relazione (E-R) si ha quando **un'entità è legata a se stessa**.

Invece di collegare due entità distinte (come STUDENTE e CORSO), la relazione esprime un legame tra **due diverse occorrenze della stessa entità**.

Per capire se una relazione ricorsiva è **simmetrica** o **non simmetrica**, si osserva il comportamento dei **ruoli** e il verso della relazione.

1. I Ruoli nelle Relazioni Ricorsive

Poiché l'entità coinvolta è una sola, è fondamentale specificare i **ruoli** che le due occorrenze assumono all'interno del legame.

- **Esempio Classico**: Entità PERSONA, relazione SPOSATO_CON. Entrambi i partecipanti hanno il ruolo di "Coniuge".
- **Esempio Gerarchico**: Entità IMPIEGATO, relazione SUPERVISIONA. Un'occorrenza ha il ruolo di "Manager", l'altra ha il ruolo di "Subordinato".

2. Relazione Simmetrica vs Non Simmetrica

La distinzione nasce da questa domanda: *se l'elemento A è legato all'elemento B, significa automaticamente che anche B è legato ad A nello stesso identico modo?*

A. Relazione Simmetrica

Una relazione è simmetrica quando il legame **vale in entrambi i versi** con lo stesso significato. I ruoli giocati dalle due occorrenze sono equivalenti e intercambiabili.

- **Esempio tipico**: Entità PERSONA con la relazione SPOSATO_CON.
  - Se Mario è sposato con Anna, allora Anna è automaticamente sposata con Mario.
  - **Ruoli**: Entrambi sono "Coniuge".
- **Cardinalità tipica**: Spesso `(0,1)` a `(0,1)` (monogamia).
- **Proprietà**: Non c'è una direzione o una gerarchia.

B. Relazione Non Simmetrica (Asimmetrica / Orientata)

Una relazione è non simmetrica quando il legame ha un **verso ben preciso** e i due ruoli sono distinti. Se A è legato a B, B **non** è legato ad A nello stesso modo.

- **Esempio gerarchico**: Entità IMPIEGATO con la relazione SUPERVISIONA.
  - Se Mario (Manager) supervisiona Luca (Subordinato), Luca **non** supervisiona Mario.
  - **Ruoli**: Un'occorrenza è "Supervisore", l'altra è "Supervisionato".

- **Esempio di propedeuticità**: Entità `CORSO` con la relazione `PROPEDEUTICO_A`.
  - Se *Analisi 1* è propedeutico ad *Analisi 2*, *Analisi 2* non è propedeutico ad *Analisi 1*.
  - **Ruoli**: "Propedeutico" e "Successivo".
- **Cardinalità tipica**: Dipende dal contesto, ad esempio `(0,1)` lato supervisore e `(0,N)` lato subordinati.

Riassunto Strutturato

Tipo di Relazione Ricorsiva	Ruoli delle Occorrenze	Orientamento del Legame	Esempio Tipico
Simmetrica	Identici per entrambe le parti	Bidirezionale (senza verso)	`PERSONA` sposata con `PERSONA`
Non Simmetrica	Distinti e asimmetrici (es. A e B)	Unidirezionale / Gerarchico	`IMPIEGATO` supervisiona `IMPIEGATO`

Ecco la trascrizione ripulita, strutturata e integrata con tutte le spiegazioni richieste sugli argomenti di **Informatica e Modellazione Database (Diagrammi E-R)**.

Modello E-R: Attributi, Cardinalità e Identificatori Esterni

1. Attributi Multivalore e Opzionali

Un attributo può assumere più valori per una singola occorrenza (es. *Numero di telefono* di una persona).

Per modellare un attributo come **opzionale** (cioè un campo che può non essere valorizzato, come `NULL` nei database) si utilizza la cardinalità dell'attributo:

- **(0, 1)**: Attributo opzionale e a valore singolo (può non esserci o esserci al massimo una volta).
- **(1, 1)**: Attributo obbligatorio a valore singolo.
- **(0, N) / (1, N)**: Attributi multivalore opzionali o obbligatori.

Nota: Se una cardinalità minima è 0, l'attributo/partecipazione è **opzionale**; se la cardinalità minima è 1, è **obbligatoria**.

2. Spiegazione Approfondita: Identificatore Esterno

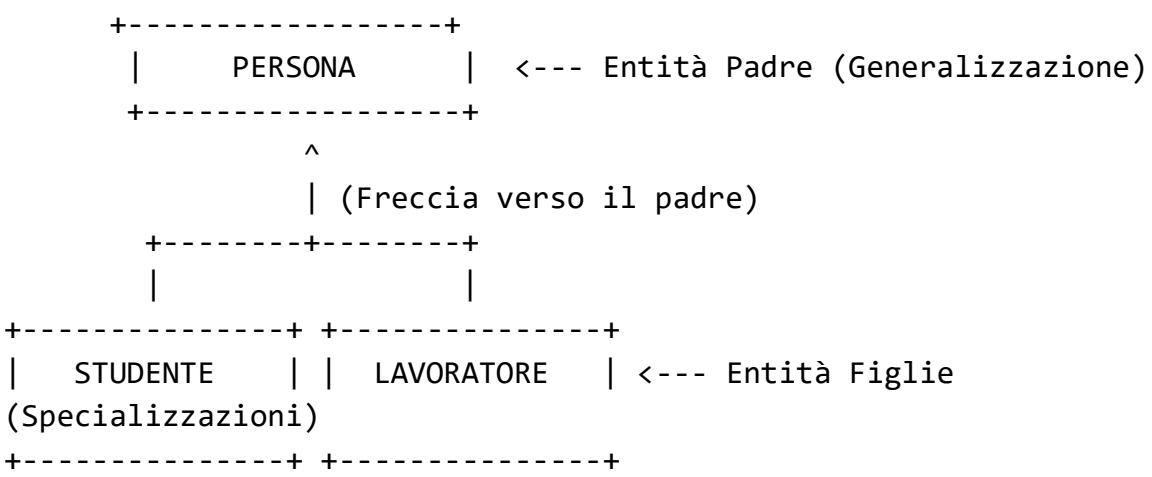
L'**identificatore esterno** (o chiave esterna a livello concettuale) permette di identificare in modo univoco un'istanza di un'entità *debole* attraverso il legame (associazione) con un'altra entità detta *forte*.

- **Regola fondamentale della cardinalità (1,1):** Un identificatore esterno può esistere **solo se la cardinalità della relazione per l'entità debole è esattamente (1,1)**.
- **Perché?** L'entità debole dipende interamente dall'entità forte. Se un'entità debole non avesse un legame univoco e obbligatorio (1,1) con l'entità forte, non si potrebbe garantire l'identificazione certa delle sue occorrenze basandosi sulla chiave dell'entità padre.

Le Generalizzazioni nei Diagrammi E-R

Una **generalizzazione** rappresenta un legame logico tra un'entità "padre" (più generale) e una o più entità "figlie" (più specifiche, dette anche **specializzazioni**).

- **Notazione grafica:** Si disegna con una freccia in grassetto (o una linea spessa/freccia vuota) che parte dalle entità figlie e **punta verso l'entità padre**.
- **Relazione di inclusione:** E_1 è generalizzazione di E_2, E_3, \dots, E_n. Viceversa, E_2, E_3 sono specializzazioni di E_1.
- **Ereditarietà (analogia con la programmazione OOP):** Come nelle classi della programmazione ad oggetti, la specializzazione eredita **tutti gli attributi e le relazioni** dell'entità padre.

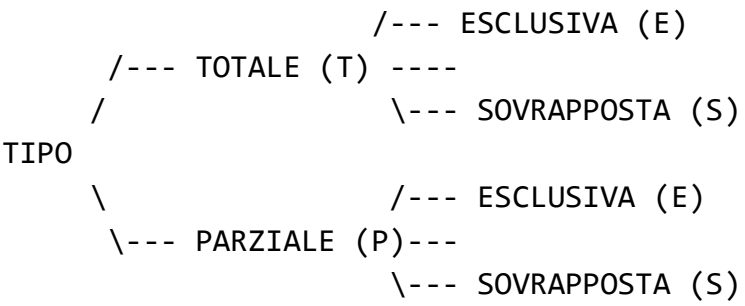


Regole per la Definizione degli Attributi e delle Chiatte

1. **Attributi comuni sul Padre:** Gli attributi generali (es. *Nome*, *Cognome*, *Codice Fiscale*) vanno definiti **solo** sull'entità padre. **Riscriverli sulle entità figlie è un errore grave di modellazione**.
2. **Attributi specifici sui Figli:** Nelle entità figlie si inseriscono unicamente le proprietà esclusive di quel sottotipo (es. *Matricola* per *Studente*, *Stipendio* o *Mansione* per *Lavoratore*).
3. **Identificatore Univoco (Chiave):** È sufficiente posizionare l'identificatore primario sull'entità padre. Esso verrà ereditato automaticamente da tutte le entità figlie.

Proprietà delle Generalizzazioni: Copertura e Sovrapposizione

Le generalizzazioni si classificano in base a due dimensioni: **Copertura** (Totale vs Parziale) e **Esclusività** (Esclusiva vs Sovrapposta).



Spiegazione delle 4 Combinazioni di Generalizzazione

1. Totale Esclusiva (T, E)

- **Definizione:** Ogni occorrenza dell'entità padre appartiene **a una ed una sola** delle entità figlie. Non esistono occorrenze del padre fuori dalle figlie e un'istanza non può stare in più figlie contemporaneamente.
- **Esempio:** L'entità PERSONA si divide in UOMO o DONNA. Ogni persona deve essere per forza o uomo o donna (Totale) e non può essere entrambe le cose (Esclusiva).

2. Totale Sovrapposta (T, S)

- **Definizione:** Ogni occorrenza dell'entità padre deve appartenere ad **almeno una** delle entità figlie, ma può appartenere a **più di una** contemporaneamente.
- **Esempio:** In un contesto universitario, una PERSONA deve essere registrata o come STUDENTE o come DOCENTE (Totale: non ci sono utenti generici), ma un docente può anche essere iscritto a un corso ed essere contemporaneamente STUDENTE (Sovrapposta).

3. Parziale Esclusiva (P, E)

- **Definizione:** Un'occorrenza dell'entità padre può non appartenere ad alcuna entità figlia (Parziale), ma se vi appartiene, può stare in **al più una** figlia (Esclusiva).
- **Esempio:** PERSONA specializzata in STUDENTE e `Lavoratore`. Esistono persone che non sono né studenti né lavoratori (es. pensionati, disoccupati → Parziale), e se una persona è inserita nel sistema, il modello prevede che o è solo studente o solo lavoratore (Esclusiva).

4. Parziale Sovrapposta (P, S)

- **Definizione:** Un'occorrenza del padre può non appartenere a nessuna figlia, oppure può appartenere a **una o più** entità figlie contemporaneamente.
- **Esempio:** PERSONA specializzata in STUDENTE e LAVORATORE. Una persona può essere solo cittadino generico (Parziale), oppure può essere contemporaneamente sia STUDENTE che LAVORATORE (Sovrapposta).

Scelte di Modellazione: Gerarchie multiple vs Attributi

Durante la progettazione si incontrano spesso casi in cui si può scegliere tra creare una sotto-specializzazione o usare un semplice attributo.

- **Esempio:** Modellare la differenza tra studente *Triennale* e *Magistrale*.
  - **Soluzione A (Attributo):** Entità STUDENTE con un attributo `TipoCorso` (`"Triennale"` / `"Magistrale"`).
  - **Soluzione B (Specializzazione):** Entità STUDENTE con due entità figlie `STUDENTE_TRIENNALE` e `STUDENTE_MAGISTRALE`.

Come scegliere?

Non esiste una risposta univoca:

- Usare un **Attributo** è preferibile se la distinzione serve solo a classificare e non ci sono attributi o relazioni differenti tra le due categorie.
- Usare la **Specializzazione** è necessario se le due categorie hanno **attributi specifici distinti** o partecipano a **relazioni diverse** nel sistema.

Impatto sulle prestazioni nel Passaggio al Modello Relazionale (SQL):

- Mantenere tutto su una sola entità/tabella semplifica le query (niente operazioni costose di `JOIN`).
- Creare più tabelle specializzate riduce lo spazio vuoto (valori `NULL`), ma richiede di eseguire dei `JOIN` o delle `UNION` quando occorre selezionare tutti gli elementi dell'entità padre.

Classifica la generalizzazione. Per ogni situazione, indica se la specializzazione è totale o parziale ed esclusiva o sovrapposta. Motiva ogni
risposta con un'unione o un'intersezione.

1. Mezzi di trasporto: ogni mezzo considerato è terrestre oppure aereo; nessun mezzo appartiene a
entrambi i gruppi.
2. Dipendenti di un'azienda: ogni dipendente è impiegato o dirigente; alcuni possono ricoprire entrambi
i ruoli.
3. Contenuti di una piattaforma: ogni contenuto è un video o un articolo; alcuni contenuti della
piattaforma sono podcast e non appartengono a nessuno dei due gruppi. Un contenuto non può essere
sia video sia articolo.
4. Persone iscritte a un centro sportivo: una persona può essere iscritta a nuoto, pallavolo, a entrambi
oppure a nessuno dei due corsi.

La Documentazione di Supporto allo Schema E-R

Il diagramma E-R, per quanto chiaro, da solo **non è sufficiente** a esprimere tutte le sfumature e le logiche di un sistema informativo. Per questo motivo viene affiancato da una **documentazione di supporto**, la quale ha tre scopi fondamentali:

1. **Facilitare l'interpretazione** dello schema concettuale.
2. **Descrivere concetti e proprietà** che non possono essere rappresentati direttamente nel diagramma grafico (come i domini specifici o i valori ammissibili).
3. **Definire le Regole Aziendali (*Business Rules*)**, distinguendo tra vincoli di integrità e regole di derivazione.

1. Descrizione dei Concetti e Proprietà Specifiche

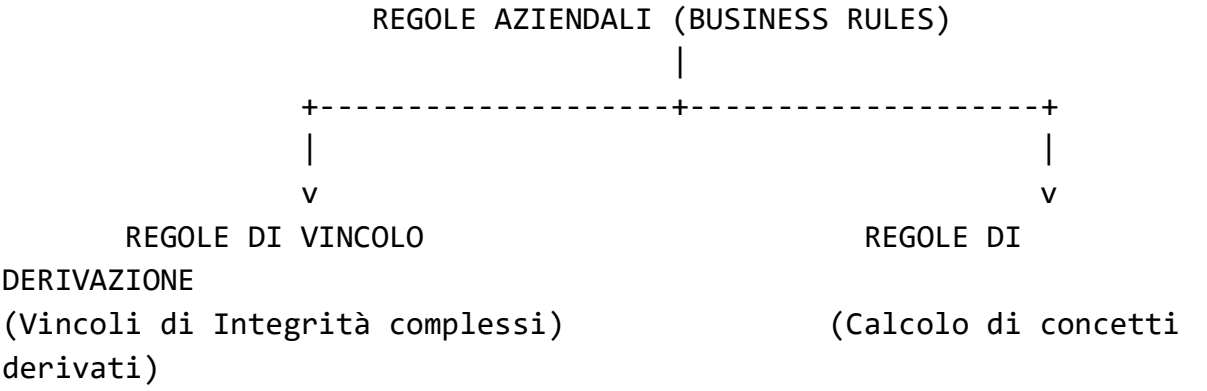
Molti attributi hanno regole sui valori ammissibili che non sono visibili a livello di puro grafico E-R.

- **Esempio di Vincolo di Dominio:** Un attributo come `StatoLibro` all'interno dell'entità `LIBRO` non è un semplice testo generico, ma deve assumere solo valori appartenenti a un insieme predefinito (dominio enumerato):

$$\text{StatoLibro} \in \{\text{'Attivo'}, \text{'Passivo'}, \text{'In corso'}\}$$
Questo tipo di specifiche viene documentato nel **Dizionario dei Concetti** (o Dizionario dei Dati), affiancato alle definizioni narrative di ciascuna Entità e Relazione.

2. Le Regole Aziendali (*Business Rules*)

Le **Business Rules** servono a formalizzare tutte le logiche di business non espresse dal modello grafico. Si dividono principalmente in due categorie:



A. Regole di Vincolo (Vincoli di Integrità)

Impongono condizioni che i dati devono **sempre rispettare** per essere considerati validi all'interno del database.

- *Esempio: "Un impiegato non può partecipare a più di 5 progetti contemporaneamente"* oppure *"La data di inizio di un progetto deve essere antecedente alla data di fine"*.

B. Regole di Derivazione (Concetti Calcolati)

Definiscono come calcolare o ricavare il valore di un attributo/concetto a partire da altre operazioni su concetti presenti nello schema.

- **Struttura formale:**

$$\langle \text{concetto} \rangle \text{ si ottiene } \langle \text{operazioni su concetti} \rangle$$

- **Esempio pratico:**

*"Il **budget** di un **progetto** si ottiene moltiplicando per tre la somma degli **stipendi** degli **impiegati** che vi partecipano."*

$$\text{Budget Progetto} = 3 \times \left(\sum \text{Stipendi degli impiegati partecipanti} \right)$$

3. Implementazione nel Database: SQL, Procedure e Trigger

Mentre le definizioni generali (entità, attributi, chiavi primarie, cardinalità $(0,1)$, $(1,N)$) si traducono direttamente nelle tabelle SQL tramite i costrutti standard (`CREATE TABLE`, `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `CHECK`), le **Business Rules più complesse e le Regole di Derivazione** necessitano di strumenti SQL avanzati:

1. **Trigger:** Procedure automatiche scatenate prima o dopo un evento di inserimento, modifica o cancellazione (`INSERT`, `UPDATE`, `DELETE`). Consentono di bloccare transazioni che violano le regole di vincolo aziendali o di aggiornare automaticamente attributi derivati.
2. **Stored Procedure / Funzioni:** Blocchi di codice SQL lato server eseguiti per calcolare attributi derivati complessi (es. il calcolo dinamico del budget di un progetto) prima di restituire i dati all'applicazione.

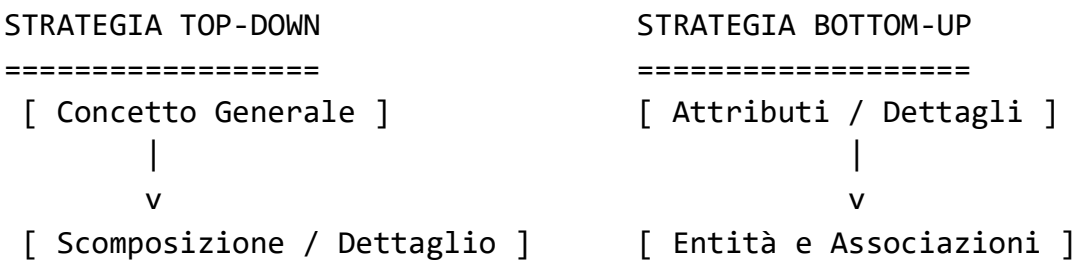
Quadro Riassuntivo della Progettazione Concettuale

Con l'aggiunta della documentazione di supporto, il pacchetto finale della **Progettazione Concettuale** comprende:

Componente	Contenuto e Scopo
Diagramma E-R Grafico	Struttura visiva composta da Entità, Attributi, Relazioni, Cardinalità e Generalizzazioni.
Dizionario dei Concetti	Descrizione testuale di Entità, Relazioni e vincoli sui singoli Attributi (es. domini come `StatoLibro`).
Regole di Vincolo	Formalizzazione dei vincoli di integrità complessi non rappresentabili graficamente.
Regole di Derivazione	Formule e algoritmi per il calcolo dei concetti derivati.

1. Le Strategie di Progettazione Concettuale

Per sviluppare un diagramma E-R a partire dai requisiti del problema esistono diverse **strategie di progettazione**. Le due principali e contrapposte sono la strategia **Top-Down** e la strategia **Bottom-Up**.



2. Strategia Top-Down (Dall'Alto verso il Basso)

Cos'è?

Si parte da una visione globale e astratta del sistema (macro-concetti), per poi espandere, raffinare e scomporre man mano gli elementi in dettagli più specifici.

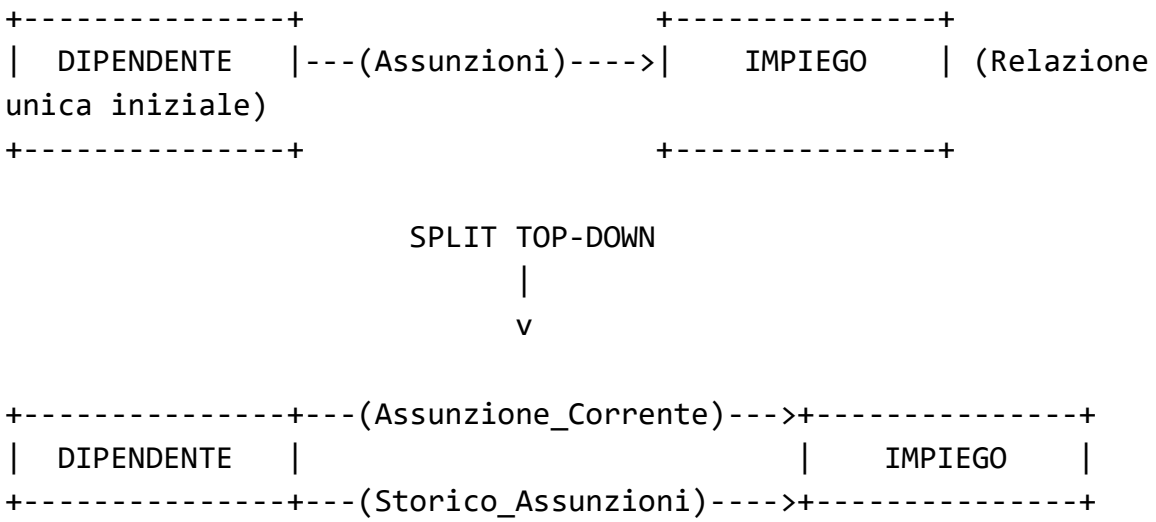
Trasformazioni Top-Down Tipiche:

1. **Raffinamento da Entità a Relazione tra Entità:**
    a. Si parte da una singola entità generale A e la si trasforma/espande in due entità distinte E_1 ed E_2 collegate da una relazione R.
    b. *Esempio:* Inizialmente si considera solo il concetto di `CORSO`. Successivamente lo si raffina dividendolo in `CORSO` e `TIPOLOGIA_CORSO`, legati dalla relazione *Appartiene*.

2. **Scomposizione di un'Entità (Split):**
    a. Un concetto ampio viene suddiviso in più entità specializzate o correlate per modellare meglio il dominio applicativo.
3. **Scomposizione/Split di una Relazione (Gestione dello Storico):**
    a. Si trasforma una singola associazione tra due entità in **due relazioni distinte**, ad esempio per separare i **dati correnti** dai **dati storici (passati)**.

Esempio Pratico (Gestione Storico e Performance):

- Consideriamo l'associazione tra l'entità `DIPENDENTE` e `IMPIEGO` (o `DOCENTE` e `CORSO`).
- Mantenere un'unica relazione *Assunzione* per 50 anni accumula una mole enorme di dati, appesantendo le interrogazioni frequenti.
- **Soluzione Top-Down:** Splittare la relazione in due:
    - **Relazione 1 (`Assunzione_Corrente`):** Registra solo la posizione lavorativa/docenza attiva dell'anno in corso (dati leggeri e veloci da consultare).
    - **Relazione 2 (`Storico_Assunzioni`):** Conserva le registrazioni dei contratti e degli anni passati.



4. **Aggiunta Progressiva degli Attributi:**
    a. Si definiscono prima tutte le entità e le associazioni principali dello schema. In un secondo momento (passo Top-Down) si specificano gli attributi interni e i dettagli di ciascuna entità o relazione.

Vantaggi e Svantaggi della Strategia Top-Down

- **Vantaggi:** Permette di focalizzarsi subito sulla struttura fondamentale del database senza perdersi nei dettagli fin dall'inizio.
- **Svantaggi:** È applicabile principalmente se si possiede già una visione globale e astratta di tutte le componenti del sistema.

3. Strategia Bottom-Up (Dal Basso verso l'Alto)

Cos'è?

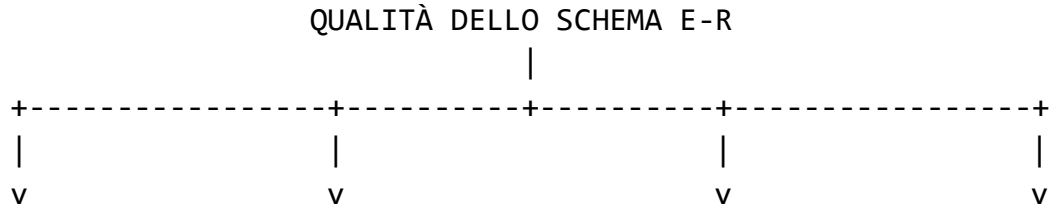
Si parte dai dettagli elementari (attributi o singole occorrenze) per poi aggregarli progressivamente in strutture più complesse come entità e generalizzazioni.

Esempi di Trasformazioni Bottom-Up:

1. **Collegamento tra concetti esistenti:**
    a. Si individuano prima le singole entità (es. `STUDENTE` e `CORSO`) e successivamente ci si chiede qual è il legame tra loro, introducendo la relazione (es. *Iscrizione*).
2. **Generalizzazione (Bottom-Up):**
    a. Si parte da entità specifiche già definite (es. `STUDENTE_TRIENNALE` e `STUDENTE_MAGISTRALE`) e si nota che condividono proprietà ed attributi comuni. Si crea quindi un'entità padre sovrastante (`STUDENTE`) su cui muovere le caratteristiche condivise.
3. **Dai Dati/Attributi alle Entità:**
    a. Si elenca prima l'insieme di tutti gli attributi necessari (es. *Nome*, *Cognome*, *Matricola*, *Codice Fiscale*, *Stipendio*) e poi li si raggruppa nelle rispettive entità appartenenti.

4. Parametri di Valutazione e Qualità dello Schema E-R

Come negli algoritmi non esiste un'unica soluzione esatta, anche per un diagramma E-R possono esistere più modelli validi. Tuttavia, la qualità di uno schema concettuale si valuta su **4 parametri fondamentali**:



1. CORRETTEZZA (Sintattica e Semantica) 2. COMPLETEZZA (Copertura di tutti i dati) 3. LEGGIBILITÀ (Chiarezza del grafico) 4. MINIMALITÀ (Assenza di ridondanze)

1. Correttezza

- **Correttezza Sintattica:** Rispetto rigoroso delle regole e dei simboli del formalismo E-R (es. usare i rettangoli per le entità, i rombi per le relazioni, le linee con pallino per gli attributi; non dimenticare i pallini o fare confusione tra i simboli).
- **Correttezza Semantica:** Utilizzo dei concetti in modo coerente con il loro significato reale (es. non usare una relazione per modellare un concetto che in realtà è un'entità o una generalizzazione).

2. Completezza

- Lo schema deve rappresentare **tutti i dati di interesse** espressi nel testo dei requisiti e deve consentire di eseguire tutte le operazioni e interrogazioni richieste dall'utente/cliente senza perdite di informazione.

3. Leggibilità

- Il diagramma deve essere chiaro, ordinato e visivamente comprensibile. Incroci di linee, sovrapposizioni di scritte o elementi ammassati in un angolo del foglio compromettono la qualità del progetto.

4. Minimalità

- Lo schema **non deve contenere ridondanze** o concetti duplicati.
- *Eccezione:* Se viene introdotta una ridondanza (come lo split di una relazione in storica e corrente), questa **deve essere espressamente motivata** da un'esigenza prestazionale o logica ben definita.